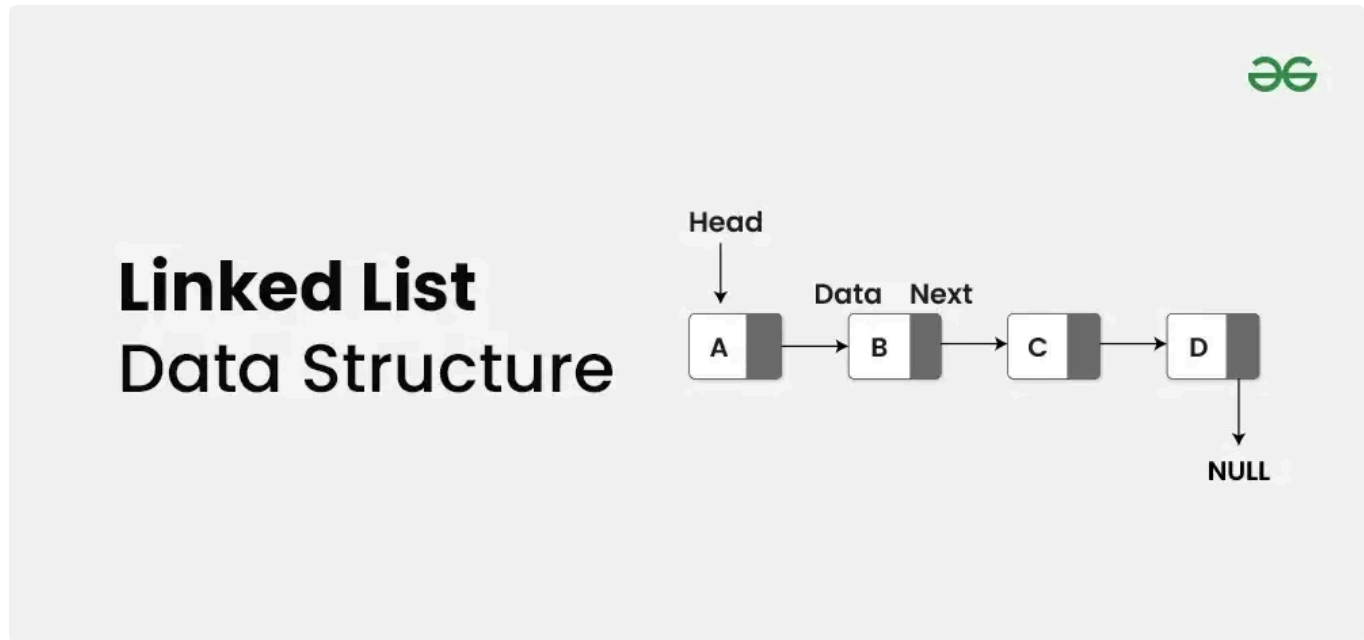


26 Aug, 2026

Linked Lists — Complete Notes

1. What is a Linked List?



A linked list is a linear data structure where elements ("nodes") are stored in memory at *non-contiguous* locations. Each node holds:

1. **Data** — the actual value
2. **Pointer(s)** — address of the next (and possibly previous) node

Unlike arrays, elements are not accessed by index directly — you must traverse from a known starting point (the **head**).

Why use it over an array?

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous
Access time	$O(1)$ random access	$O(n)$ — must traverse
Insertion/Deletion at start	$O(n)$ (shifting)	$O(1)$
Size	Fixed (static array)	Dynamic
Extra memory	None	Pointer overhead per node

2. Types of Linked Lists

- **Singly Linked List (SLL):** each node points to the next node only.
 - **Doubly Linked List (DLL):** each node has a pointer to next *and* previous node.
 - **Circular Linked List (CLL):** the last node points back to the head instead of `NULL` . Can be singly or doubly circular.
-

3. Node Structure (C++)

```
#include <iostream>
using namespace std;

// Singly Linked List Node
struct Node {
    int data;
    Node* next;
};

// Doubly Linked List Node
struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};
```

No `std::vector` , no templates — plain pointers and `struct` s, as is standard for DSA-level implementation.

4. Core Operations & Algorithms

For each operation: algorithm (numbered steps), time complexity, and C++ code.

4.1 Traversal

Goal: visit every node once, typically to print or process data.

Algorithm:

1. Set a pointer `temp = head` .

2. While `temp != NULL` : a. Process/print `temp->data` . b. Move forward: `temp = temp->next` .
3. Stop when `temp == NULL` .

Time Complexity: $O(n)$ | **Space:** $O(1)$

```
void traverse(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
```

4.2 Searching

Goal: find whether a value `key` exists, and where.

Algorithm:

1. Set `temp = head` , `position = 0` .
2. While `temp != NULL` : a. If `temp->data == key` , return `position` (found). b. Else, `temp = temp->next` , `position = position + 1` .
3. If loop ends, return `-1` (not found).

Time Complexity: $O(n)$ | **Space:** $O(1)$

```
int search(Node* head, int key) {
    Node* temp = head;
    int pos = 0;
    while (temp != NULL) {
        if (temp->data == key) return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}
```

4.3 Insertion

Three cases: at the beginning, at the end, and at a given position.

(a) Insert at Beginning

Algorithm:

1. Create a new node `newNode` .
2. Set `newNode->data = value` .
3. Set `newNode->next = head` .
4. Set `head = newNode` (update head to the new node).

Time Complexity: $O(1)$

```
Node* insertAtHead(Node* head, int value) {  
    Node* newNode = new Node();  
    newNode->data = value;  
    newNode->next = head;  
    head = newNode;  
    return head;  
}
```

(b) Insert at End

Algorithm:

1. Create `newNode` with `newNode->next = NULL` .
2. If `head == NULL` , set `head = newNode` and stop.
3. Else, traverse: set `temp = head` , move `temp` until `temp->next == NULL` .
4. Set `temp->next = newNode` .

Time Complexity: $O(n)$ (or $O(1)$ if a tail pointer is maintained)

```
Node* insertAtEnd(Node* head, int value) {  
    Node* newNode = new Node();  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if (head == NULL) return newNode;  
  
    Node* temp = head;  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

```
    return head;
}
```

(c) Insert at Given Position (0-indexed)

Algorithm:

1. If `position == 0`, insert at beginning (case a).
2. Else, traverse to node just before the target position: move `temp` (`position - 1`) times.
3. Create `newNode`.
4. Set `newNode->next = temp->next`.
5. Set `temp->next = newNode`.

Time Complexity: $O(n)$

```
Node* insertAtPosition(Node* head, int value, int position) {
    if (position == 0) return insertAtHead(head, value);

    Node* temp = head;
    for (int i = 0; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }
    if (temp == NULL) return head; // invalid position

    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
```

4.4 Deletion

(a) Delete from Beginning

Algorithm:

1. If `head == NULL`, list is empty, stop.
2. Store `temp = head`.
3. Move `head = head->next`.
4. Free `temp`.

Time Complexity: $O(1)$

```
Node* deleteAtHead(Node* head) {
    if (head == NULL) return NULL;
    Node* temp = head;
    head = head->next;
    delete temp;
    return head;
}
```

(b) Delete from End

Algorithm:

1. If `head == NULL` , stop.
2. If only one node (`head->next == NULL`), delete `head` , set `head = NULL` .
3. Else, traverse until `temp->next->next == NULL` (i.e., `temp` is second-last node).
4. Delete `temp->next` , set `temp->next = NULL` .

Time Complexity: $O(n)$

```
Node* deleteAtEnd(Node* head) {
    if (head == NULL) return NULL;
    if (head->next == NULL) {
        delete head;
        return NULL;
    }
    Node* temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = NULL;
    return head;
}
```

(c) Delete by Value / Key

Algorithm:

1. If `head == NULL` , stop.
2. If `head->data == key` , delete head (case a), stop.

3. Else, set `temp = head` . Move `temp` until `temp->next->data == key` or `temp->next == NULL` .
4. If `temp->next == NULL` , key not found, stop.
5. Else, store `toDelete = temp->next` , set `temp->next = toDelete->next` , free `toDelete` .

Time Complexity: $O(n)$

```
Node* deleteByValue(Node* head, int key) {
    if (head == NULL) return NULL;
    if (head->data == key) return deleteAtHead(head);

    Node* temp = head;
    while (temp->next != NULL && temp->next->data != key) {
        temp = temp->next;
    }
    if (temp->next == NULL) return head; // not found

    Node* toDelete = temp->next;
    temp->next = toDelete->next;
    delete toDelete;
    return head;
}
```

4.5 Reversal

Goal: reverse the direction of all `next` pointers so the last node becomes head.

Algorithm (Iterative, 3-pointer technique):

1. Initialize `prev = NULL` , `curr = head` .
2. While `curr != NULL` : a. Store `nextNode = curr->next` (save before breaking link). b. Reverse link: `curr->next = prev` . c. Move pointers forward: `prev = curr` , `curr = nextNode` .
3. Set `head = prev` (prev is the new head after loop ends).

Time Complexity: $O(n)$ | **Space:** $O(1)$

```
Node* reverseList(Node* head) {
    Node* prev = NULL;
    Node* curr = head;
    while (curr != NULL) {
        Node* nextNode = curr->next;
```

```

        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }
    return prev;
}

```

Recursive version:

```

Node* reverseRecursive(Node* head) {
    if (head == NULL || head->next == NULL) return head;
    Node* newHead = reverseRecursive(head->next);
    head->next->next = head;
    head->next = NULL;
    return newHead;
}

```

4.6 Detecting a Cycle (Floyd's Cycle Detection / "Tortoise and Hare")

Goal: determine if the list loops back on itself instead of ending in `NULL`.

Algorithm:

1. Initialize two pointers: `slow = head`, `fast = head`.
2. While `fast != NULL` and `fast->next != NULL`:
 - a. Move `slow = slow->next` (one step).
 - b. Move `fast = fast->next->next` (two steps).
 - c. If `slow == fast`, a cycle exists — return true.
3. If loop exits normally (`fast` hit `NULL`), no cycle — return false.

Why it works: if there's a cycle, the faster pointer eventually laps the slower one inside the loop; they must meet.

Time Complexity: $O(n)$ | **Space:** $O(1)$

```

bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
}

```

```
    return false;
}
```

4.7 Finding the Middle Node (Fast-Slow Pointer)

Algorithm:

1. `slow = head` , `fast = head` .
2. While `fast != NULL` and `fast->next != NULL` : a. `slow = slow->next` b. `fast = fast->next->next`
3. When loop ends, `slow` is at the middle node.

Time Complexity: $O(n)$ | **Space:** $O(1)$

```
Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

4.8 Merging Two Sorted Linked Lists

Goal: combine two sorted lists into one sorted list.

Algorithm:

1. Create a dummy node to simplify edge cases; `tail = dummy` .
2. While both `l1` and `l2` are not `NULL` : a. If `l1->data <= l2->data` , attach `l1` to `tail->next` , move `l1 = l1->next` . b. Else, attach `l2` , move `l2 = l2->next` . c. Move `tail = tail->next` .
3. Attach whichever list is left (`l1` or `l2`) to `tail->next` .
4. Return `dummy->next` (skip the dummy).

Time Complexity: $O(n)$

```

Node* mergeSorted(Node* l1, Node* l2) {
    Node dummy;
    Node* tail = &dummy;
    dummy.next = NULL;

    while (l1 != NULL && l2 != NULL) {
        if (l1->data <= l2->data) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }
        tail = tail->next;
    }
    tail->next = (l1 != NULL) ? l1 : l2;
    return dummy.next;
}

```

4.9 Removing Duplicates (from a Sorted List)

Algorithm:

1. Set temp = head .
2. While temp != NULL and temp->next != NULL : a. If temp->data == temp->next->data , skip the duplicate: store dup = temp->next , set temp->next = temp->next->next , delete dup . b. Else, move temp = temp->next .

Time Complexity: $O(n)$

```

Node* removeDuplicates(Node* head) {
    Node* temp = head;
    while (temp != NULL && temp->next != NULL) {
        if (temp->data == temp->next->data) {
            Node* dup = temp->next;
            temp->next = temp->next->next;
            delete dup;
        } else {
            temp = temp->next;
        }
    }
}

```

```
    return head;
}
```

5. Doubly Linked List — Key Differences

Same operations as SLL, but every insertion/deletion must also fix the `prev` pointer.

Insert at Beginning (DLL):

1. Create `newNode`, set `newNode->data = value`.
2. Set `newNode->prev = NULL`, `newNode->next = head`.
3. If `head != NULL`, set `head->prev = newNode`.
4. Update `head = newNode`.

```
DNode* insertAtHeadDLL(DNode* head, int value) {
    DNode* newNode = new DNode();
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;
    if (head != NULL) head->prev = newNode;
    head = newNode;
    return head;
}
```

Advantage: can traverse backward, and deletion of a known node is $O(1)$ (no need to find the previous node — you already have `node->prev`).

6. Circular Linked List — Key Idea

- Last node's `next` points to `head` instead of `NULL`.
- Traversal must use a `do-while` loop or check `temp->next == head` as the stopping condition (not `NULL`).

```
void traverseCircular(Node* head) {
    if (head == NULL) return;
    Node* temp = head;
    do {
        cout << temp->data << " ";
        temp = temp->next;
    }
```

```
    } while (temp != head);  
    cout << endl;  
}
```

Use cases: round-robin scheduling, circular buffers, multiplayer turn systems.

7. Complexity Summary Table

Operation	Singly Linked List	Doubly Linked List	Array
Access by index	$O(n)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$
Insert at head	$O(1)$	$O(1)$	$O(n)$
Insert at tail	$O(n)$ (or $O(1)$ with tail ptr)	$O(1)$ (with tail ptr)	$O(1)$ amortized
Delete at head	$O(1)$	$O(1)$	$O(n)$
Delete known node	$O(n)$ (need prev)	$O(1)$	$O(n)$
Reverse	$O(n)$	$O(n)$	$O(n)$
Extra space/node	1 pointer	2 pointers	none

8. Where Linked Lists Show Up in Data Science / Interviews

- **Underlying structure** for stacks, queues, and adjacency lists in graphs.
 - **Hash map chaining** — collisions in a hash table are often resolved using linked lists.
 - **LRU Cache** implementation uses a doubly linked list + hash map.
 - Common interview problems: reverse a list, detect/remove cycle, merge k sorted lists, find nth node from end, palindrome check on a list.
-

9. Quick Revision Checklist

- ☐ Can you write traversal, search, insert (3 cases), delete (3 cases) from scratch?
- ☐ Do you know the 3-pointer reversal technique by heart?
- ☐ Can you explain Floyd's cycle detection and prove why slow/fast pointers must meet?
- ☐ Do you understand why DLL deletion is $O(1)$ but SLL deletion is $O(n)$ for a known node?

☐ Can you handle edge cases: empty list, single node, deleting head/tail?